

Expense App Offline-First Architecture

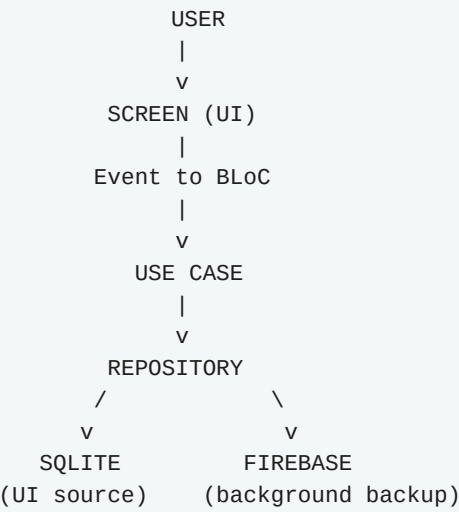
Clean Architecture + BLoC + GetIt + SQLite + Firebase

Login requires internet.

Everything else is local-first with silent background sync.

1. Big Picture

- Rule 1: UI always reads from SQLite/local.
- Rule 2: Save, Update, Delete write to local first.
- Rule 3: Firebase sync runs in background; user sees no cloud loading.



2. Clean Architecture Layers

```
SCREEN
| context.read<PropertyBloc>().add(SaveEvent)
v
BLOC
| await savePropertyUseCase(...)
v
USE CASE
| business rules
v
REPOSITORY INTERFACE
|
v
REPOSITORY IMPLEMENTATION
|-----|
v           v
LOCAL DATASOURCE    REMOTE DATASOURCE
SQLite              Firebase
```

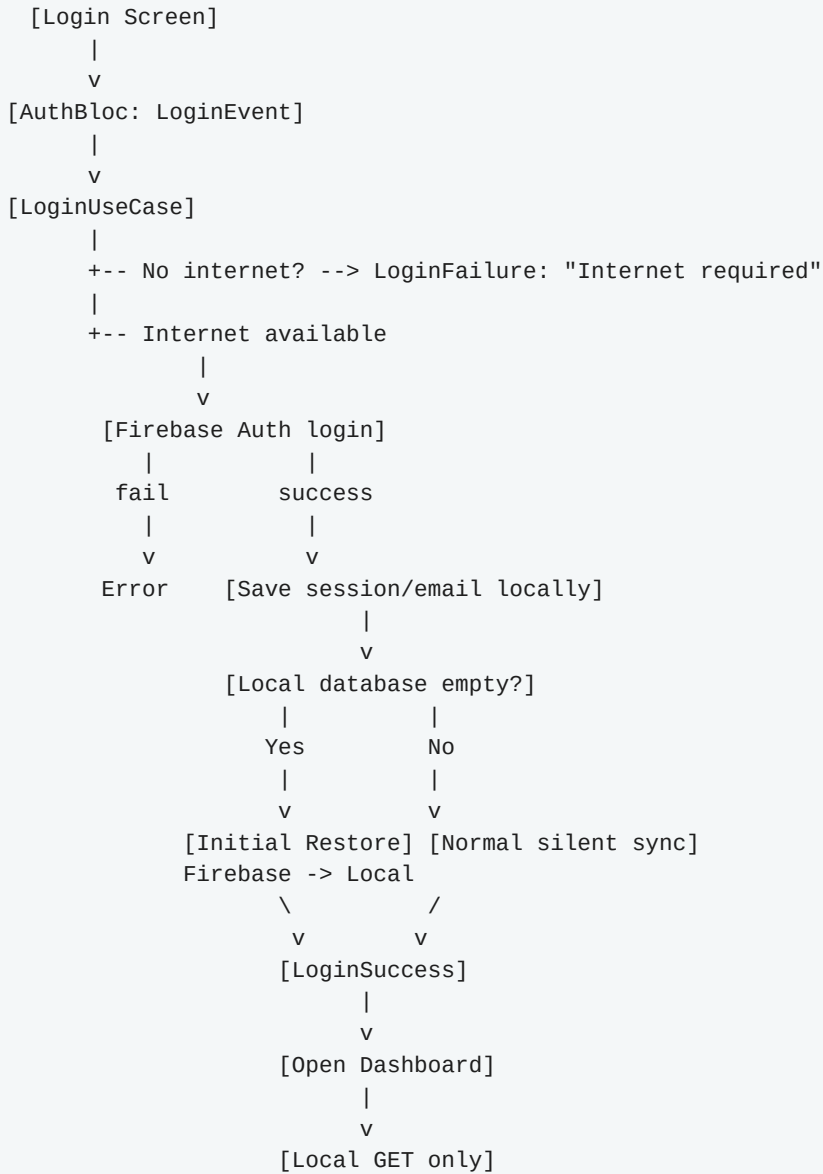
Where to work when adding a feature

UI button / screen	-> Screen + BLoC Event/State
Business rule	-> UseCase
SQLite query	-> Local DataSource
Firestore query	-> Remote DataSource
Connect both	-> Repository
Background sync	-> SyncService
Register dependencies	-> GetIt module

3. Recommended Folder Structure

```
lib/
|-- core/
|   |-- database/db_helper.dart
|   |-- network/internet_checker.dart
|   |-- sync/sync_service.dart
|   `-- di/get_it.dart + modules/
|
`-- features/
    |-- auth/
    |   |-- data/
    |   |-- domain/
    |   `-- presentation/
    |
    `-- property/
        |-- data/
        |   |-- datasources/
        |   |   |-- property_local_datasource.dart
        |   |   `-- property_remote_datasource.dart
        |   `-- repositories/property_repository_impl.dart
        |-- domain/
        |   |-- entities/property_entity.dart
        |   |-- repositories/property_repository.dart
        |   `-- usecases/
        `-- presentation/
            |-- bloc/
            `-- screens/
```

4. Login Flow (Internet Required)



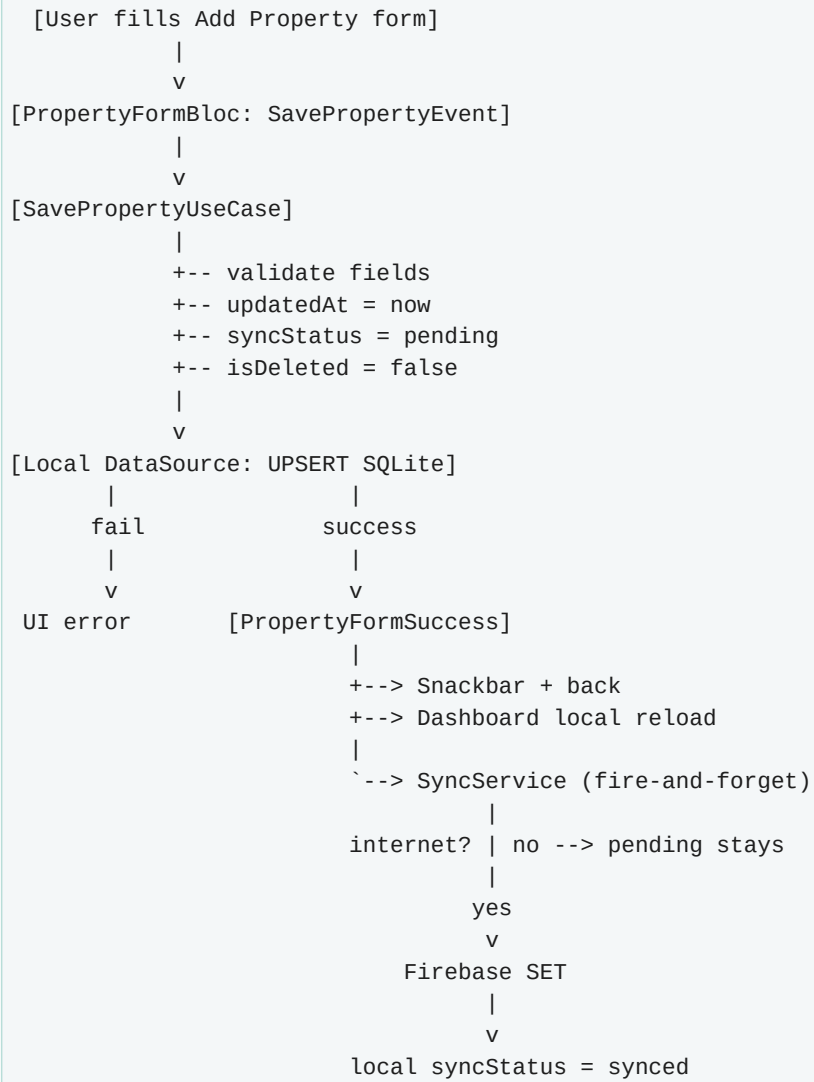
5. Dashboard GET Flow (Local Only)

```
[Dashboard opens]
  |
  v
[PropertyListBloc: LoadPropertiesEvent]
  |
  v
[GetPropertiesUseCase]
  |
  v
[AuthLocal: current user email/id]
  |
  v
[PropertyRepository.getLocalProperties]
  |
  v
[SQLite SELECT where user = current user and isDeleted = 0]
  |
  +-- Empty list --> "No Property Added" + Add button
  |
  `-- Data -----> Property cards immediately
```

Parallel background flow

```
Dashboard already showing local data
  |
  v
[SyncService - silent]
  |
  internet available?
  |           |
  No          Yes
  |           |
  stop       push + pull + merge
           |
           v
        SQLite update
           |
           v
optional silent BLoC reload
```

6. Save Flow (Local First)



7. Update Flow

```
[User opens Edit]
  |
  v
[Existing entity loaded into form]
  |
  v
[UpdatePropertyEvent]
  |
  v
[UpdatePropertyUseCase]
  |
  +-- same stable cardId
  +-- updatedAt = now
  +-- syncStatus = pending
  |
  v
[SQLite UPSERT]
  |
  +--> UI success + local refresh
  |
  `--> Background Firebase SET/UPDATE
        |
        v
      mark local synced
```

8. Delete Flow (Soft Delete)

```
[Delete tap]
  |
  v
[Confirmation dialog]
  |
  v
[DeletePropertyEvent]
  |
  v
[DeletePropertyUseCase]
  |
  +-- isDeleted = true
  +-- syncStatus = pending
  +-- updatedAt = now
  |
  v
[Hide item from local list]
  |
  v
[Background SyncService]
  |
  v
[Firestore document delete]
  |
  v
[Optional local hard delete]
```


9. Reinstall / New Phone Restore

```
    OLD INSTALL
Local SQLite: data available
Firebase:      synced backup
    |
    v
[App uninstall]
    |
    +--> Local SQLite deleted
    `--> Firebase data remains safe

NEW INSTALL
[User logs in with same account]
    |
    v
[Firebase Auth success]
    |
    v
[Save current user email/id locally]
    |
    v
[Initial Restore UseCase]
    |
    v
[Firebase get all properties]
    |
    v
[Local bulk UPSERT, status = synced]
    |
    v
[Dashboard Local GET]
    |
    v
[Old data appears again]
```

10. Background Sync Algorithm

```
SYNC START
|
+-- Internet unavailable? --> STOP silently
|
|-- Internet available
    |
    v
STEP 1: PUSH LOCAL PENDING
    |
    +-- pending save/update --> Firebase SET
    +-- pending delete -----> Firebase DELETE
    +-- success -----> local status = synced
    |-- failure -----> pending/failed; retry later
    |
    v
STEP 2: PULL REMOTE
    |
    v
For each remote record:
    local missing? -----> insert local
    remote.updatedAt > local? -----> remote overwrites local
    local pending and newer? -----> keep local; push next
    timestamps equal? -----> do nothing
    |
    v
STEP 3: SILENT UI REFRESH (optional)
```

Sync triggers

- After login success
- After save / update / delete
- When internet reconnects
- When app resumes
- Optional periodic worker

11. BLoC Responsibilities

AuthBloc

LoginSubmitted	-> LoginUseCase	-> LoginLoading / Success / Failure
LogoutRequested	-> LogoutUseCase	-> LogoutSuccess / Failure

PropertyListBloc

LoadProperties	-> GetPropertiesUseCase	-> PropertyLoaded / PropertyError
LocalDataChanged	-> local reload only	-> PropertyLoaded

PropertyFormBloc

SaveProperty	-> SavePropertyUseCase	-> Saved / failure
UpdateProperty	-> UpdatePropertyUseCase	-> Updated / failure
DeleteProperty	-> DeletePropertyUseCase	-> Deleted / failure

IMPORTANT:

BLoC must never call Firebase directly.

12. GetIt Registration Map

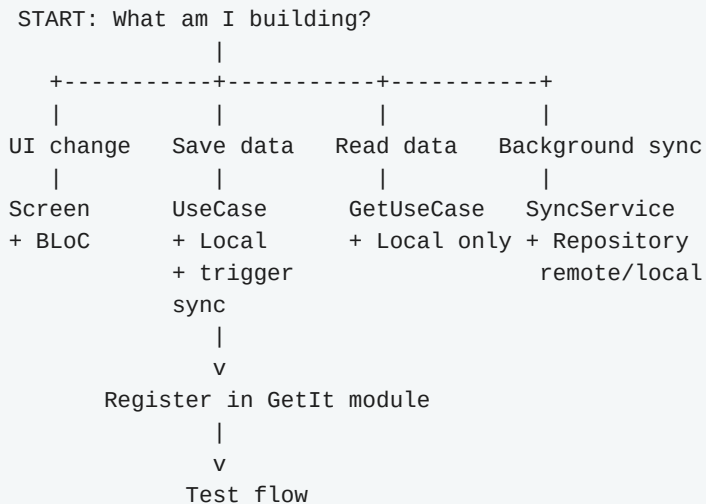
SINGLETONS

- DBHelper
- InternetChecker
- AuthLocalDataSource
- AuthRemoteDataSource
- PropertyLocalDataSource
- PropertyRemoteDataSource
- AuthRepository
- PropertyRepository
- SyncService
- LoginUseCase
- GetPropertiesUseCase
- SavePropertyUseCase
- UpdatePropertyUseCase
- DeletePropertyUseCase
- RestoreFromCloudUseCase

FACTORIES

- AuthBloc
- PropertyListBloc
- PropertyFormBloc

13. Decision Tree - What Should I Build Now?



14. Beginner Implementation Order

PHASE 1: Foundation

1. Entity
2. SQLite table
3. Local DataSource
4. Remote DataSource
5. Repository interface + impl

PHASE 2: Business

6. GetPropertiesUseCase
7. SavePropertyUseCase
8. UpdatePropertyUseCase
9. DeletePropertyUseCase
10. RestoreFromCloudUseCase
11. SyncService

PHASE 3: UI

12. PropertyListBloc + Dashboard Screen
13. PropertyFormBloc + Add/Edit Screen

PHASE 4: Auth

14. LoginUseCase
15. AuthBloc + Login Screen

PHASE 5: Wire

16. GetIt modules
17. Router
18. Test each flow

15. Complete Add Property Trace

Save button

- > PropertyFormBloc.add(SavePropertyEvent)
- > SavePropertyUseCase
- > PropertyRepository.saveLocal
- > PropertyLocalDataSource.upsert
- > SQLite success
- > BLoC emits success
- > Screen closes
- > Dashboard Bloc loads local list

At the same time:

SavePropertyUseCase

- > SyncService.sync (not awaited by UI)
- > Firebase set
- > Local markSynced

16. Quick Reference Card

LOGIN -> Internet + Firebase Auth + local session + restore
DASHBOARD -> Local GET only
SAVE -> Local pending -> background Firebase
UPDATE -> Local upsert pending -> background Firebase
DELETE -> Local soft delete pending -> background Firebase
REINSTALL -> Login -> Firebase restore -> local display
UI -> Screen -> BLoC -> UseCase
SYNC -> Push pending + pull remote + merge by updatedAt
GETIT -> BLoCs factory; services/repos lazy singleton

Internet rules

Action	Internet	User sees

Login / Sign up	Required	Button loading + auth error
Dashboard / Reports	Not required	Local data instantly
Add / Edit / Delete	Not required	Instant local success
Firebase sync	When available	Silent background work
Reinstall restore	At login	Old data comes back

Testing checklist

- [] First login with empty local and empty Firebase
- [] Online add -> local immediately + Firebase eventually
- [] Offline add -> local immediately + pending status
- [] Internet reconnect -> pending item uploads automatically
- [] Offline update -> UI shows latest local value
- [] Offline delete -> item hidden, cloud delete on reconnect
- [] Firebase timeout -> dashboard still shows local data
- [] App restart offline -> local data available
- [] Uninstall/reinstall + same login -> remote data restored
- [] Different user login -> previous user's local data hidden
- [] Two devices update -> updatedAt merge rule works
- [] Repeated sync -> no duplicates